Predictive Maintenance

Interim Project Report

Harshit Rathi

Engine condition classification | 2 November 2026

Executive overview

The supplied dataset contains 19,535 observations and six sensor measurements. I use these measurements to examine whether the two recorded Engine Condition classes can be distinguished. I first check the file and set aside a test split, then explore the training data and compare a tuned random forest with a majority-class baseline.

On 3,907 held-out observations, the random forest achieved 0.65 balanced accuracy and 0.80 average precision for class 1, versus 0.50 and 0.63 for the baseline.

The healthy/faulty mapping remains unverified because external sources conflict. This report retains class 0/class 1 terminology throughout. Class 1 is the positive class solely for statistical evaluation; it does not imply healthy, faulty, or better condition.

The result is evidence of predictive association within this file. There is no time horizon, equipment identifier or prospective failure history, so advance warning of a breakdown and production readiness have not been established.

Business objective

The practical aim is to help a maintenance analyst decide which engine observations need closer review. A useful system would find concerning conditions without sending too many normal observations for inspection. For this interim study, I can assess how well the recorded classes are separated. Estimating inspection workload or savings will require confirmed label definitions and the costs of each kind of error.

1. Data registration and background

Methodology

I load the CSV from the repository data folder and check the seven expected columns, numeric sensor values, binary target, missing entries, duplicates and finite values. I also record the file checksum (SHA-256), so a later run can be checked against the same input.

Source: Parv Modi, Automotive Vehicles Engine Health Dataset, Kaggle, version 1 (2023). The user-supplied engine_data.csv was verified byte-for-byte against the public download on 2 November 2026. Collection protocol and pressure/temperature units remain unverified.

The healthy/faulty mapping remains unverified because external sources conflict. Nafar describes class 1 as healthy, while Chen describes class 1 as requiring maintenance. The original source metadata does not resolve this disagreement. See Appendix A for references. No class has been relabeled or interpreted as better/worse condition.

Collection background: the supplied file is a tabular snapshot. No documentation establishes where or when engines were measured, how the condition label was assigned, whether rows belong to repeated engines, or whether the observations were simulated. These gaps limit generalization and physical interpretation; no provenance is inferred from the filename.

Table 1. Registered dataset overview

Check	Result
Rows / columns	19,535 / 7
Predictors / outcome	Six numeric sensors / Engine Condition (0 or 1)
Class 0	7,218 (36.95%)
Class 1	12,317 (63.05%)
Missing values / exact duplicate rows	0 / 0
Unnecessary / leakage fields	No identifier or outcome-derived field is evident; all six sensors retained
Units and label definitions	rpm is explicit; pressure/temperature units and label meanings unconfirmed

Schema: Engine rpm, Lub oil pressure, Fuel pressure, Coolant pressure, lub oil temp, Coolant temp, Engine Condition.

The raw upload is preserved as engine_data.csv; the validated analysis copy is data/engine_data.csv. Their checksum is recorded in Appendix A. No external images or replacement datasets were used.

2. Exploratory data analysis

Methodology

I restrict exploratory data analysis (EDA) to the 15,628 training observations. Summaries and histograms show the distribution of each sensor; class means and boxplots show how the groups compare; a correlation matrix examines linear relationships across variables. Keeping the 3,907 test rows aside avoids using the final evaluation data to guide these choices.

Table 2. Training sensor summary

Sensor	Mean	Std. dev.	Median	Minimum	Maximum
Engine rpm	792.26	267.34	748.00	61.00	2239.00
Lub oil pressure	3.31	1.02	3.16	0.00	7.27
Fuel pressure	6.65	2.76	6.19	0.00	21.14
Coolant pressure	2.34	1.04	2.17	0.00	7.48
lub oil temp	77.65	3.12	76.82	71.32	89.58
Coolant temp	78.43	6.22	78.34	62.45	195.53

Values shown as 0.00 are rounded small positive pressures, not zero-valued or missing readings. Units other than rpm were not supplied.

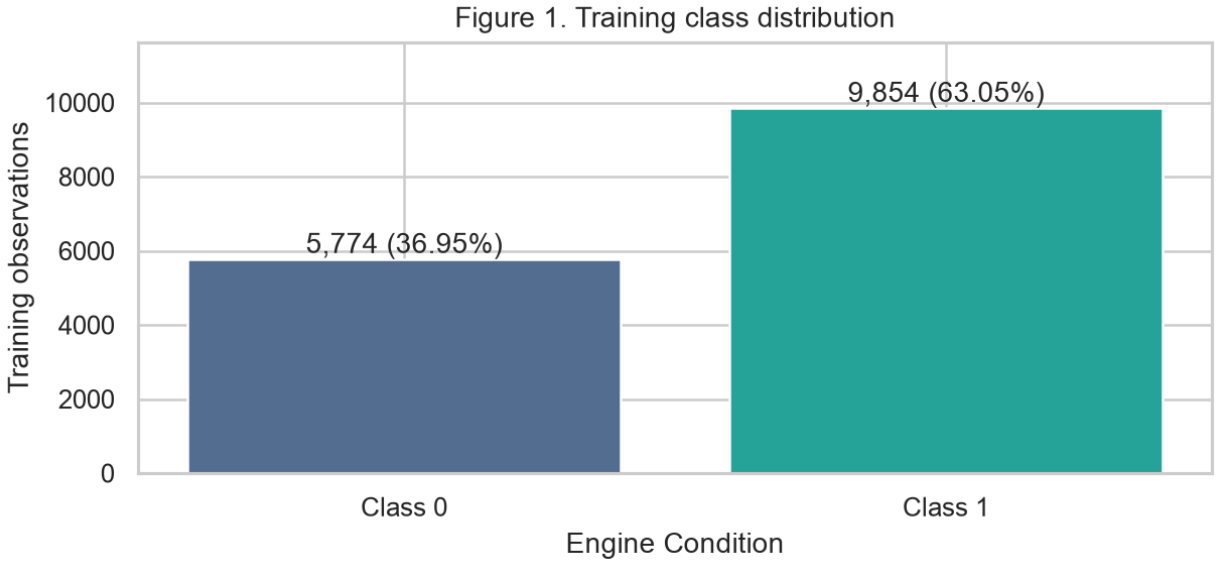


Figure 1. Class 1 is the more frequent class; the imbalance is moderate, not an extremely rare-event setting.

Class 1 represents 63.05% of training observations. A majority-class prediction can therefore appear accurate while never recognizing class 0; balanced accuracy and both-class diagnostics are needed.

2.1 Univariate patterns and unusual readings

Outlier checks use the interquartile range (IQR), the difference between the 75th and 25th percentiles. Values beyond 1.5 IQR below/above those percentiles are flagged for review.

Figure 2. Sensor distributions (training data)

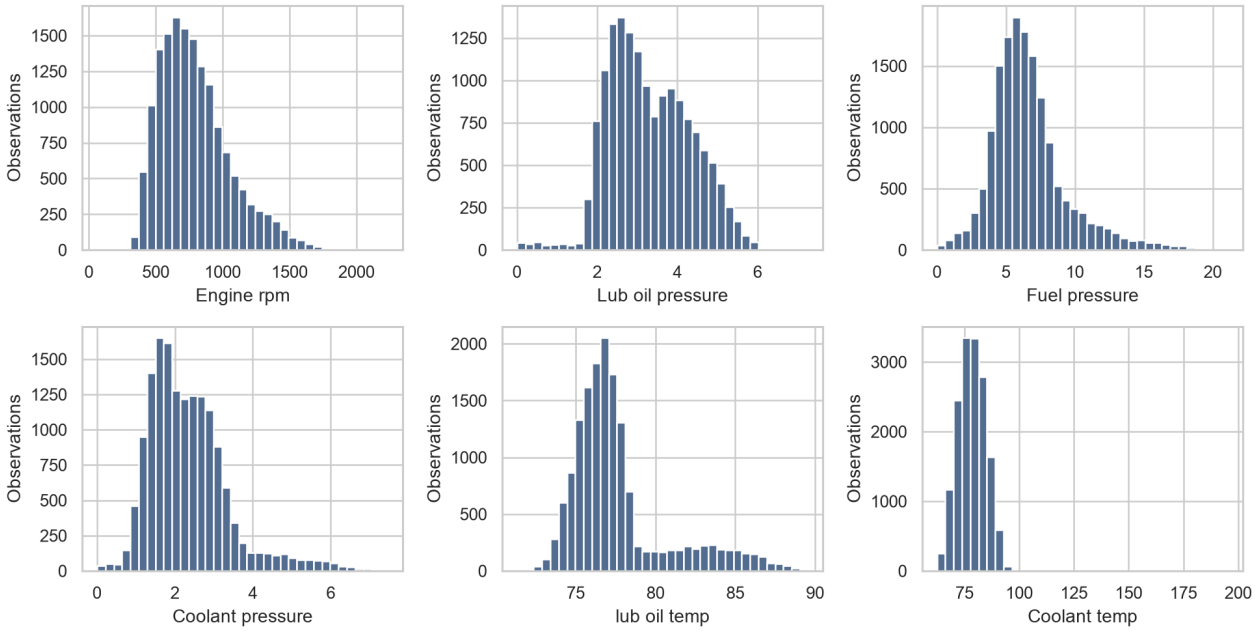


Figure 2. Histograms use the entire training range so unusually high sensor values remain visible.

Engine rpm and the pressure features have upper tails; oil temperature is concentrated near the mid-to-high 70s with a smaller high-temperature tail. A wide coolant-temperature range is driven partly by a small set of high observations, so typical operation is better described by both median and spread.

Coolant temp has a median of 78.34 and a maximum of 195.53; 2 readings lie outside its 1.5-IQR limits. Retain these readings because they may describe unusual operating conditions; verify units and sensor calibration before applying physical limits.

Table 3. Training outlier checks

Sensor	Outside 1.5-IQR limits	Handling
Engine rpm	371	Retained; no verified physical limit supplied
Lub oil pressure	52	Retained; no verified physical limit supplied
Fuel pressure	909	Retained; no verified physical limit supplied
Coolant pressure	641	Retained; no verified physical limit supplied
lub oil temp	2107	Retained; no verified physical limit supplied
Coolant temp	2	Retained; no verified physical limit supplied

Business implication: review extreme readings with sensor/domain experts. Automatically deleting them could remove the very operating states a condition model should recognize.

2.2 Bivariate class comparisons

Table 4. Mean sensor readings by class (training set)

Sensor	Class 0	Class 1	Difference (1 - 0)
Engine rpm	885.74	737.49	-148.24
Lub oil pressure	3.23	3.35	0.12
Fuel pressure	6.23	6.90	0.68
Coolant pressure	2.37	2.33	-0.04
lub oil temp	78.03	77.43	-0.60
Coolant temp	78.81	78.21	-0.60

Figure 3. Sensor distributions by class (training data)

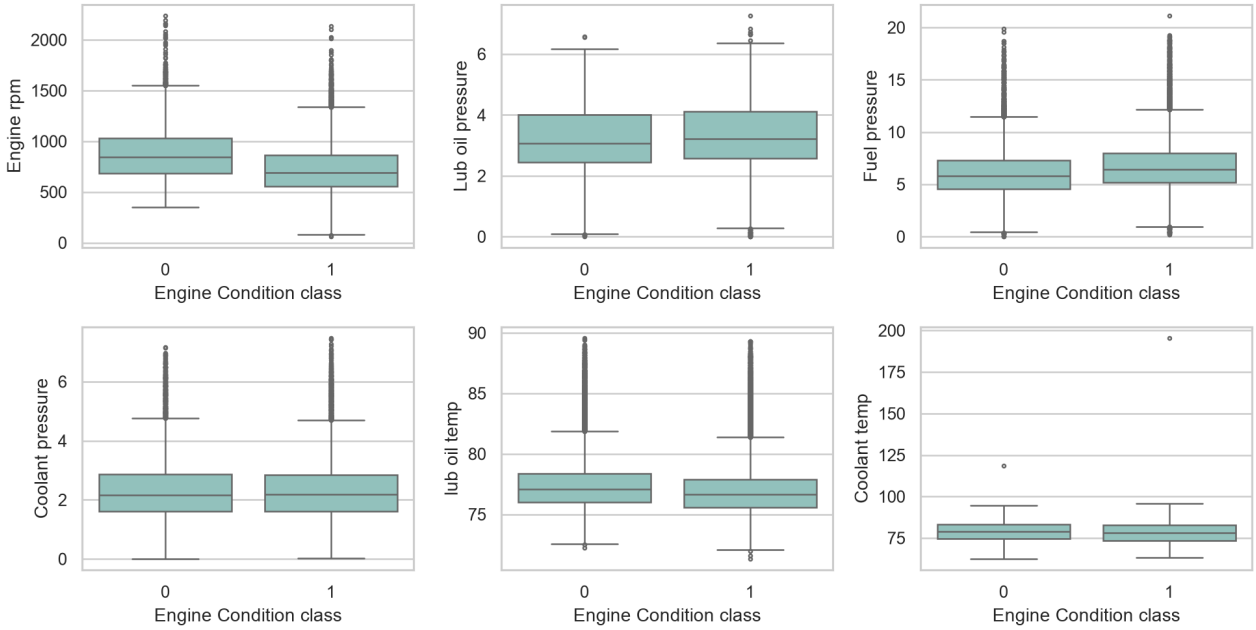


Figure 3. Each panel compares one sensor with Engine Condition; overlap limits single-sensor rules.

Engine rpm averages 885.74 for class 0 and 737.49 for class 1. The distributions overlap, so rpm alone should not be used as a maintenance rule.

Class 1 also has higher mean fuel pressure and lower mean oil temperature in this sample. These differences support testing a combined sensor model, but their direction cannot be translated into “better” or “worse” condition until the label definitions are verified.

2.3 Multivariate associations and implications

Figure 4. Multivariate Pearson correlations (training data)

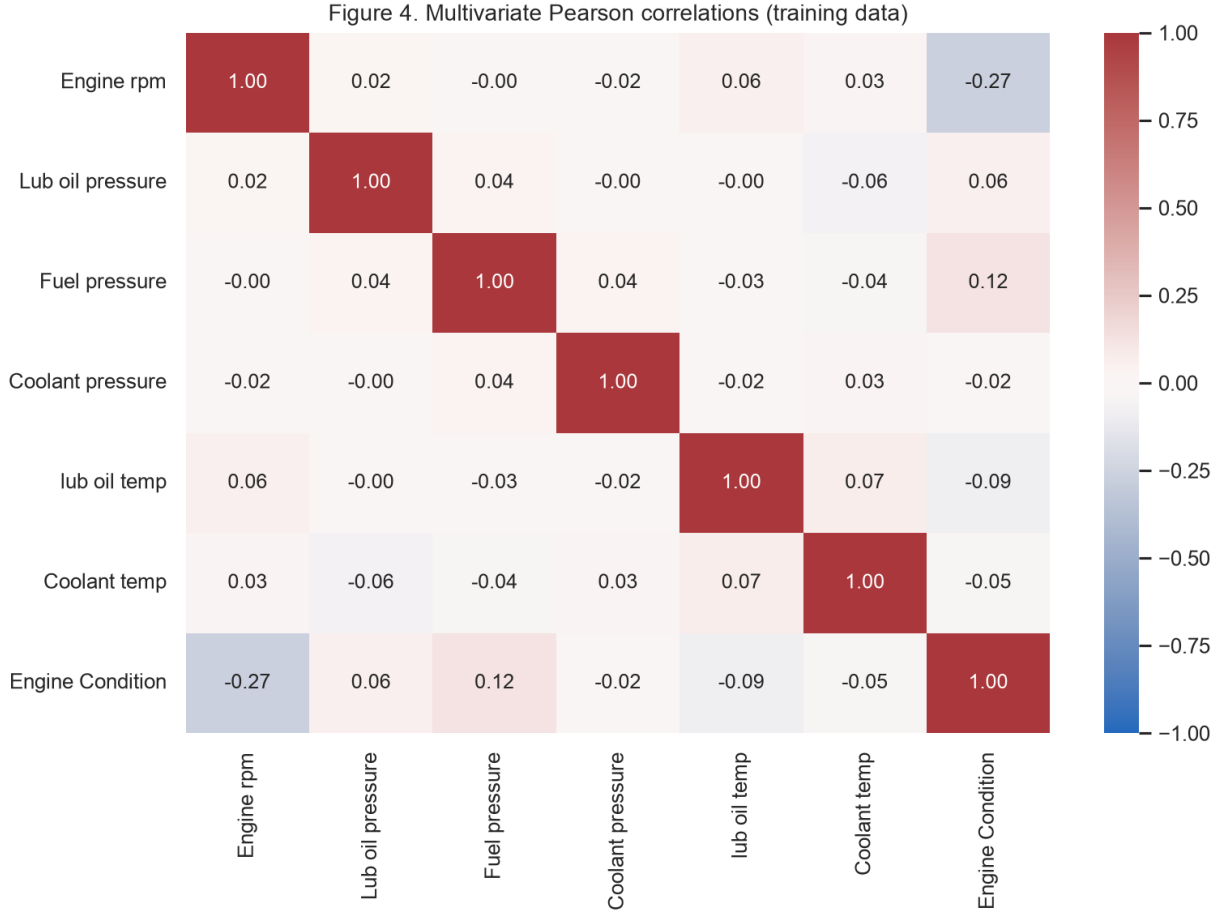


Figure 4. Pearson correlations summarize linear relationships; encoded target correlation is interpreted as a class association.

Engine rpm has the strongest absolute linear association with the target (r = -0.27). This describes an association with the encoded class, not a cause of engine condition.

The strongest sensor-to-sensor correlation is lub oil temp versus Coolant temp (r = 0.07). Weak pairwise correlations do not rule out nonlinear relationships or interactions.

Business implication: there is no strong linear redundancy that justifies dropping a sensor on correlation grounds alone. A tree ensemble can test nonlinear combinations while retaining the available measurement context. Correlation does not demonstrate causation.

No identifier, timestamp, or failure-mode field is present. All six sensor features are retained. These measurements support contemporaneous condition classification; the data cannot establish advance warning time or remaining useful life.

3. Data preparation

Methodology

I retain all six sensors and check whether identical predictor rows carry conflicting labels. The preparation function removes exact duplicates if any are found and saves a stratified 80/20 split using seed 42. Stratification keeps similar class proportions in both splits. No rows need removal or missing-value replacement in this file. Median imputation stays inside the fitted pipeline as a safeguard and is learned separately within each training fold.

Table 5. Preparation decisions and rationale

Step	Decision and reason
Column removal	None. There are no IDs or explicit post-outcome columns in the supplied schema. Prediction-time sensor availability still needs confirmation.
Duplicate / missing handling	No missing values or exact duplicates observed. No rows removed. Conflicting identical predictor rows are rejected.
Extreme values	Retained. No documented physical ranges justify deletion or capping.
Scaling / encoding	No scaling needed for the tree model; all observed predictors are numeric. The reusable pipeline also supports categorical encoding.
Training / test rows	15,628 / 3,907; stratified, random_state = 42.
Feature engineering	Use the six original readings for an interpretable baseline. No unsupported unit-dependent ratios or time features are invented.
Leakage prevention	No target used as a predictor; imputation fit within folds; hyperparameter selection excludes test data.

Saved artifacts: artifacts/splits/train.csv, test.csv, registration.json and split_metadata.json. The metadata includes source/split hashes and the class mapping. The training job validates the hashes before loading the split artifacts.

GitHub Actions run 41421706788 completed successfully. The prepare job validated the data and uploaded the splits; train downloaded those splits, tuned the model and uploaded the results; persist-model committed the fitted model and logs to models/ (commit ba0ee8a). Run: https://github.com/harshitrathi14/capstone-predictivemaintenance/actions/runs/41421706788. The report and notebook retain the original local execution results; the hosted run has its own metrics and environment record in models/.

Split limitation: a random split assumes sufficiently independent rows. The CSV does not allow equipment-group or chronological validation. If repeated-engine data or timestamps become available, reevaluate with group/time separation before operational use.

4. Model building and experiment tracking

Methodology

I use a random forest because it can combine nonlinear sensor patterns without feature scaling. A prior-probability dummy classifier provides a reference based on class proportions alone. I compare 16 parameter combinations across three shuffled, stratified cross-validation (CV) folds (48 fits), selecting the highest mean validation average precision (AP) for class 1. The selected pipeline is refitted on the complete training split and evaluated on the held-out rows at the default threshold of 0.5.

Table 6. Hyperparameter search space

Parameter	Values
n_estimators	100, 200
max_depth	Unrestricted, 10
min_samples_leaf	1, 3
class_weight	None, balanced
Fixed settings	random_state = 42; model n_jobs = 1; grid n_jobs = 4

Average precision measures ranking of class 1 across thresholds. Its no-skill reference is the class-1 prevalence. Balanced accuracy averages recall for both classes, so a majority-only classifier scores 0.50. Area under the receiver operating characteristic curve (ROC AUC) measures ranking discrimination. Precision is the fraction of predicted class-1 rows actually in class 1; recall is the fraction of actual class-1 rows found. F1 is their harmonic mean. These last three metrics describe the selected 0.5 operating point.

Selected model: 100 trees, maximum depth 10, minimum 1 observation(s) per leaf, and balanced class weighting. Best mean cross-validation average precision: 0.79 (fold standard deviation 0.01). The corresponding mean training average precision is 0.93. The higher training score suggests some overfitting remains, even after limiting tree depth. The leading candidates are close; the scores do not establish that the winner would always be best on another sample.

Table 7. Top five candidates by mean validation average precision

Trees	Depth	Leaf min.	Weights	CV AP	Train AP
100	10	1	balanced	0.79	0.93
200	10	1	balanced	0.79	0.93
200	10	3	balanced	0.79	0.92
200	10	3	None	0.79	0.92
100	10	1	None	0.79	0.93

All candidate parameters, fold scores, training scores, ranks and timing are logged in experiments.csv and displayed in the HTML notebook. The saved best_model.joblib contains preprocessing and the fitted forest. best_parameters.json and run_metadata.json record full settings, environment versions and input lineage. The original local fitted model is retained in artifacts/model/. The separately trained GitHub model and its run records are saved in models/.

4.1 Held-out evaluation

Table 8. Held-out performance (positive class = 1)

Metric	Prior baseline	Random forest
Average Precision	0.63	0.80
Roc Auc	0.50	0.69
Balanced Accuracy	0.50	0.65
Precision	0.63	0.76
Recall	1.00	0.63
F1	0.77	0.69

Figure 6. Held-out ranking performance (positive class = 1)

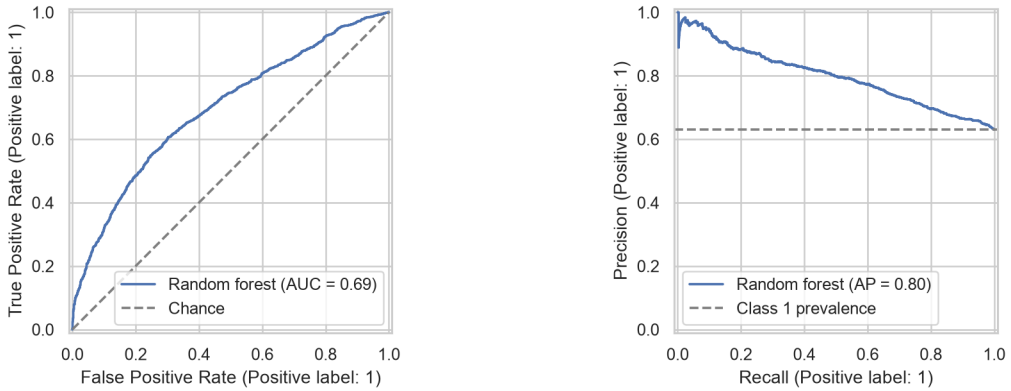


Figure 6. The diagonal ROC line and prevalence-level precision line are reference baselines. Curves describe the held-out sample only.

The test set contains 63.04% class-1 observations. Average precision improves by 0.17 over the prior baseline. This indicates additional ranking information beyond the class proportions, while remaining specific to the supplied sample.

The baseline predicts class 1 for every row because class 1 is the majority. Its perfect class-1 recall is therefore not evidence of useful separation; it recognizes none of class 0.

At threshold 0.5, the forest's class-1 F1 is 0.69, below the baseline's 0.77. Selection optimized average precision, not F1. The forest exchanges some class-1 recall for class-0 recognition. Better ranking does not guarantee a better operating point for every metric or business cost.

4.2 Errors and operational meaning

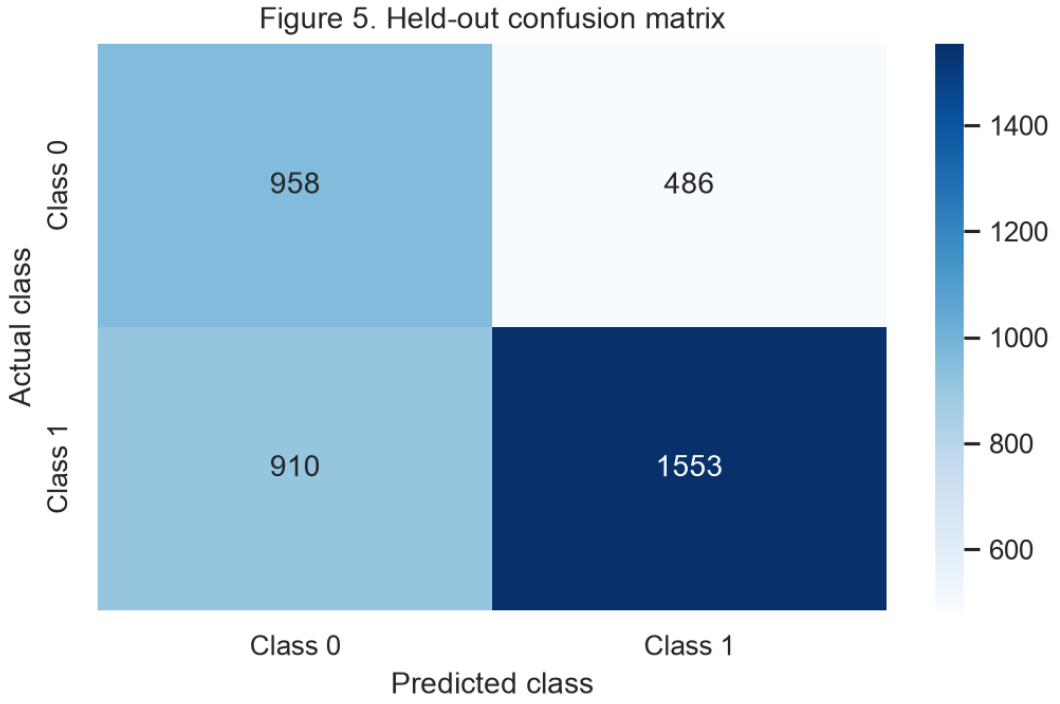


Figure 5. Rows are actual classes and columns are predicted classes; class semantics are unconfirmed.

Of 1,444 actual class-0 observations, 958 were classified as 0 and 486 as 1. Of 2,463 actual class-1 observations, 1,553 were classified as 1 and 910 as 0. Class-0 recall is 0.66; class-1 recall is 0.63.

The model predicts class 1 for 2,039 of 3,907 observations (52.19%). These predictions are not maintenance alerts. Neither error type is called a missed fault or false alarm until the healthy/faulty mapping is verified.

5. Actionable insights and next steps

1. Obtain the data dictionary first: confirm label meanings, sensor units, acquisition context and prediction-time availability. Until then, use the scores for analytical review rather than automatic maintenance orders.

2. Review extreme coolant readings and the observed rpm/class relationship with an engine specialist. Retain suspicious readings in the analysis until measurement errors can be distinguished from unusual operating states.

3. Once the intervention class is confirmed, choose a threshold on validation data using agreed costs of missed conditions and unnecessary inspections. The current threshold was not cost-optimized, and these observations do not establish financial savings.

4. Collect engine identifiers, timestamps and future failure outcomes. Evaluate on unseen engines and later periods, estimate uncertainty, and run a supervised pilot before introducing operational alerts. Deployment and monitoring are outside this interim scope.

Appendix A. Registration and reproducibility evidence

Dataset SHA-256: 616c7b985ec00e299061f02b11fd866f8eca16b5e8d15d4a61a5fbfd116b8045

Source: Parv Modi, Automotive Vehicles Engine Health Dataset, Kaggle, version 1 (2023). The user-supplied engine_data.csv was verified byte-for-byte against the public download on 2 November 2026. Collection protocol and pressure/temperature units remain unverified.

Source verification and conflicting label definitions

Original dataset: Parv Modi, Automotive Vehicles Engine Health Dataset, Kaggle, version 1 (2023). The downloaded CSV exactly matches the supplied file. The retrieved source metadata does not define the target-class meanings.

https://www.kaggle.com/datasets/parvmodi/automotive-vehicles-engine-health-dataset

Hossein Nafar, master thesis, University of Wuppertal (2024), printed page 30 / PDF page 41. Describes class 1 as healthy and class 0 as out of operation or failed; reference 47 cites the matching Kaggle dataset.

https://www.vzu.uni-wuppertal.de/fileadmin/site/vzu/Masterarbeit_HosseinNafar.pdf#page=41

Jayden P. Chen, Fusion of Neural Networks and Logistic Regression for Predictive Maintenance of Vehicle Engines (2024), Table 1, page 66. Describes class 1 as indicating a need for maintenance. Its sensor summary statistics match this dataset, although its stated row count differs.

https://ajosr.org/wp-content/uploads/journal/published_paper/volume-2/issue-2/ajsr2024_pFcUxi32.pdf#page=3

The source download and uploaded CSV have identical SHA-256 hashes. The cited disagreement prevents a verified healthy/faulty interpretation; class 0/class 1 labels are retained throughout.

Code, analysis and plots were generated for this project. Software libraries used: pandas, NumPy, scikit-learn, Matplotlib, Seaborn, joblib, Jupyter nbformat/nbconvert and ReportLab. No external visual assets were used.

train CSV SHA-256: cb2ed6e952ec52c2b14d60165ba8bf9e77537e7f4c29215fe51a8712df659f78

test CSV SHA-256: 25c2dc140bd54e619db312d40906ca20cfa711027814e435bdfc74a95cd5472f

Table A1. Execution environment

Component	Version / setting
Python	3.14.7
pandas	2.3.3
numpy	2.5.4
scikit-learn	1.9.1
joblib	1.6.0
Seed	42
Candidate count	16
CV folds	3

Verification evidence: the notebook was executed end to end and exported after execution. The HTML includes full experiment output; reports/ contains the numbered plots and EDA tables. Tests cover invalid schema/labels, disjoint splits, model serialization and unseen-category handling.

Appendix B. Extended experiment log

Table B1 below shows every candidate to five decimal places so the small score differences remain visible (main-body values use two decimal places). Full fold scores, parameters and timing appear in the HTML notebook and artifacts/model/experiments.csv.

Table B1. All 16 candidate results

Trees	Depth	Leaf	Weight	CV AP	CV std.	Rank
100	None	1	None	0.77100	0.00519	16
200	None	1	None	0.77423	0.00510	14
100	None	3	None	0.78096	0.00424	12
200	None	3	None	0.78306	0.00593	9
100	10	1	None	0.79255	0.00540	5
200	10	1	None	0.79252	0.00546	6
100	10	3	None	0.79156	0.00498	8
200	10	3	None	0.79285	0.00520	4
100	None	1	balanced	0.77415	0.00406	15
200	None	1	balanced	0.77730	0.00456	13
100	None	3	balanced	0.78123	0.00571	11
200	None	3	balanced	0.78296	0.00638	10
100	10	1	balanced	0.79382	0.00506	1
200	10	1	balanced	0.79361	0.00551	2
100	10	3	balanced	0.79245	0.00458	7
200	10	3	balanced	0.79311	0.00518	3

Appendix B (continued). Fold scores and execution timing

Candidate IDs follow the row order in Table B1 and the exported experiment log. AP is average precision. Time is the mean fit time in seconds across the three folds. Detailed training-fold scores and scoring times remain available in the full HTML log.

Table B2. Validation-fold scores and mean fit times for all candidates

Candidate	Fold 1 AP	Fold 2 AP	Fold 3 AP	Train AP	Fit seconds
1	0.77357	0.76376	0.77567	1.00000	0.98
2	0.77752	0.76702	0.77813	1.00000	1.99
3	0.78330	0.77501	0.78458	0.99875	0.93
4	0.78641	0.77473	0.78804	0.99903	1.81
5	0.79475	0.78511	0.79778	0.92832	0.72
6	0.79398	0.78522	0.79835	0.92928	1.47
7	0.79267	0.78498	0.79703	0.91715	0.71
8	0.79459	0.78579	0.79816	0.91767	1.39
9	0.77787	0.76850	0.77607	1.00000	1.06
10	0.78087	0.77087	0.78017	1.00000	2.08
11	0.78449	0.77320	0.78600	0.99903	0.97
12	0.78607	0.77406	0.78873	0.99926	1.91
13	0.79606	0.78681	0.79858	0.92847	0.75
14	0.79569	0.78607	0.79907	0.92936	1.51
15	0.79441	0.78612	0.79681	0.91804	0.75
16	0.79452	0.78619	0.79863	0.91897	1.51

Appendix C. Executed analytical code

The following code cells are the exact analytical program contained in the executed notebook, with long lines wrapped for print. Use the .ipynb file or HTML for the original formatting. The report layout generator is available separately in src/deliverables.py.

Code cell 1

```
"""Reproducible binary-classification workflow; configure against the assigned dataset."""
import argparse
import hashlib
import json
import platform
from importlib.metadata import version
from pathlib import Path
import joblib
import numpy as np
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.dummy import DummyClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.impute import SimpleImputer
from sklearn.metrics import (average_precision_score, balanced_accuracy_score,
    confusion_matrix, precision_score, recall_score, f1_score, roc_auc_score)
from sklearn.model_selection import GridSearchCV, StratifiedKFold, train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder

ROOT = Path.cwd()

def save_json(path, obj):
    Path(path).parent.mkdir(parents=True, exist_ok=True)
    Path(path).write_text(json.dumps(obj, indent=2, default=str))

def load_config(path):
    cfg = json.loads(Path(path).read_text())
    for key in ['dataset', 'source', 'target', 'positive_label', 'expected_columns', 'drop_columns']:
        if key not in cfg:
            raise ValueError(f'Missing configuration field: {key}')
    if not cfg['expected_columns'] or not cfg['source']:
        raise ValueError('Provide the documented expected columns and dataset source.')
    return cfg

def validate(cfg):
    path = (ROOT / cfg['dataset']).resolve()
    if not path.is_relative_to((ROOT / 'data').resolve()):
        raise ValueError('The dataset must be loaded from the repository data folder.')
    df = pd.read_csv(path)
    if df.empty or not df.columns.is_unique:
        raise ValueError('Dataset must be nonempty with unique column names.')
    missing = set(cfg['expected_columns']) - set(df.columns)
    extra = set(df.columns) - set(cfg['expected_columns'])
    if missing or extra:
        raise ValueError(f'Schema mismatch: missing={sorted(missing)}, unexpected={sorted(extra)}')
    target = cfg['target']
    if target not in df or df[target].isna().any() or df[target].nunique() != 2:
        raise ValueError('Target must contain exactly two classes and no missing labels.')
    if cfg['positive_label'] not in df[target].unique():
        raise ValueError('positive_label is absent from the target.')
    if target in cfg['drop_columns'] or set(cfg['drop_columns']) - set(df):
        raise ValueError('Invalid drop_columns configuration.')
    for col in cfg.get('numeric_columns', []):
        df[col] = pd.to_numeric(df[col], errors='raise')
        if np.isinf(df[col].dropna()).any():
            raise ValueError(f'Infinite values in {col}')
    summary = {'rows': len(df), 'columns': list(df), 'missing': df.isna().sum().to_dict(),
               'duplicate_rows': int(df.duplicated().sum()),
               'class_counts': df[target].value_counts().to_dict(),
               'source': cfg['source'], 'sha256': hashlib.sha256(path.read_bytes()).hexdigest()}
    return df, summary

def prepare(cfg, out):
    df, summary = validate(cfg)
    save_json(out / 'registration.json', summary)
    print(json.dumps(summary, indent=2, default=str))
    # Remove IDs and post-outcome leakage fields before deduplication and splitting.
    df = df.drop(columns=cfg['drop_columns']).drop_duplicates()
    target = cfg['target']
    if len(df.columns) < 2:
        raise ValueError('No predictors remain.')
    if df.drop(columns=target).isna().all().any():
        raise ValueError('Remove entirely empty predictors in the configuration.')
    if df.groupby(list(df.drop(columns=target)), dropna=False)[target].nunique().gt(1).any():
        raise ValueError('Identical predictors have conflicting labels; resolve before splitting.')
    df[target] = (df[target] == cfg['positive_label']).astype(int)
    train, test = train_test_split(df, test_size=0.2, random_state=42, stratify=df[target])
```

```
    if train[target].value_counts().min() < 3:
        raise ValueError('Training data needs at least three observations in each class.')
    out.mkdir(parents=True, exist_ok=True)
    train.to_csv(out / 'train.csv', index=False)
    test.to_csv(out / 'test.csv', index=False)
    save_json(out / 'split_metadata.json', {'train_rows': len(train), 'test_rows': len(test),
        'removed_rows': summary['rows'] - len(df), 'seed': 42, 'target': target,
        'positive_label': cfg['positive_label'], 'drop_columns': cfg['drop_columns'],
        'source_sha256': summary['sha256'],
        'split_sha256': {name: hashlib.sha256((out / f'{name}.csv').read_bytes()).hexdigest()
                         for name in ['train', 'test']}})
    return train, test

def evaluate(model, X, y):
    pred = model.predict(X)
    prob = model.predict_proba(X)[:, 1]
    return {'average_precision': average_precision_score(y, prob), 'roc_auc': roc_auc_score(y, prob),
            'balanced_accuracy': balanced_accuracy_score(y, pred),
            'precision': precision_score(y, pred, zero_division=0),
            'recall': recall_score(y, pred, zero_division=0), 'f1': f1_score(y, pred, zero_division=0),
            'confusion_matrix': confusion_matrix(y, pred, labels=[0, 1]).tolist()}

def train_model(cfg, splits, out):
    metadata = json.loads((splits / 'split_metadata.json').read_text())
    for key in ['target', 'positive_label', 'drop_columns']:
        if metadata[key] != cfg[key]:
            raise ValueError(f'Split artifact configuration mismatch: {key}')
    for name, digest in metadata['split_sha256'].items():
        if hashlib.sha256((splits / f'{name}.csv').read_bytes()).hexdigest() != digest:
            raise ValueError(f'Split artifact checksum mismatch: {name}')
    train, test = (pd.read_csv(splits / f'{name}.csv') for name in ['train', 'test'])
    target = cfg['target']
    X, y = train.drop(columns=target), train[target]
    Xt, yt = test.drop(columns=target), test[target]
    numeric = X.select_dtypes(include='number').columns.tolist()
    categorical = [c for c in X if c not in numeric]
    preprocessing = ColumnTransformer([
        ('numeric', SimpleImputer(strategy='median'), numeric),
        ('categorical', Pipeline([('impute', SimpleImputer(strategy='most_frequent')),
            ('encode', OneHotEncoder(handle_unknown='ignore'))]), categorical)])
    model = Pipeline([('preprocessing', preprocessing),
                      ('model', RandomForestClassifier(random_state=42, n_jobs=1))])
    grid = {'model__n_estimators': [100, 200], 'model__max_depth': [None, 10],
            'model__min_samples_leaf': [1, 3], 'model__class_weight': [None, 'balanced']}
    search = GridSearchCV(model, grid, scoring='average_precision',
        cv=StratifiedKFold(n_splits=3, shuffle=True, random_state=42), n_jobs=4,
        return_train_score=True, error_score='raise')
    search.fit(X, y)
    baseline = DummyClassifier(strategy='prior').fit(X, y)
    metrics = {'baseline': evaluate(baseline, Xt, yt), 'random_forest': evaluate(search.best_estimator_,
Xt, yt),
               'best_cv_average_precision': search.best_score_, 'best_parameters': search.best_params_,
               'test_prevalence': float(yt.mean()), 'decision_threshold': 0.5}
    out.mkdir(parents=True, exist_ok=True)
    pd.DataFrame(search.cv_results_).to_csv(out / 'experiments.csv', index=False)
    save_json(out / 'metrics.json', metrics)
    save_json(out / 'best_parameters.json', search.best_estimator_.get_params(deep=True))
    save_json(out / 'run_metadata.json', {'configuration': cfg, 'splits': metadata,
        'python': platform.python_version(),
        'packages': {name: version(name) for name in ['pandas', 'numpy', 'scikit-learn', 'joblib']},
        'selection_metric': 'average_precision', 'folds': 3, 'seed': 42,
        'candidate_count': len(search.cv_results_['params'])})
    joblib.dump(search.best_estimator_, out / 'best_model.joblib')
    print(json.dumps(metrics, indent=2))
    return metrics

print("Validation and modeling functions loaded; random seed = 42.")
```

Code cell 2

```
cfg = {'dataset': 'data/engine_data.csv', 'source': 'Parv Modi, Automotive Vehicles Engine Health
Dataset, Kaggle, version 1 (2023). The user-supplied engine_data.csv was verified byte-for-byte against
the public download on 2 November 2026. Collection protocol and pressure/temperature units remain
unverified.', 'learner_name': 'Harshit Rathi', 'target': 'Engine Condition', 'positive_label': 1,
'expected_columns': ['Engine rpm', 'Lub oil pressure', 'Fuel pressure', 'Coolant pressure', 'lub oil
temp', 'Coolant temp', 'Engine Condition'], 'numeric_columns': ['Engine rpm', 'Lub oil pressure', 'Fuel
pressure', 'Coolant pressure', 'lub oil temp', 'Coolant temp'], 'drop_columns': [], 'label_note': 'The
healthy/faulty mapping remains unverified because external sources conflict. This report retains class
0/class 1 terminology throughout. Class 1 is the positive class solely for statistical evaluation; it
does not imply healthy, faulty, or better condition.', 'source_url':
'https://www.kaggle.com/datasets/parvmodi/automotive-vehicles-engine-health-dataset', 'label_evidence':
[{'title': 'Original dataset: Parv Modi, Automotive Vehicles Engine Health Dataset, Kaggle, version 1
(2023)', 'url': 'https://www.kaggle.com/datasets/parvmodi/automotive-vehicles-engine-health-dataset',
'finding': 'The downloaded CSV exactly matches the supplied file. The retrieved source metadata does not
define the target-class meanings.'}, {'title': 'Hossein Nafar, master thesis, University of Wuppertal
(2024), printed page 30 / PDF page 41', 'url': 'https://www.vzu.uni-
wuppertal.de/fileadmin/site/vzu/Masterarbeit_HosseinNafar.pdf#page=41', 'finding': 'Describes class 1 as
healthy and class 0 as out of operation or failed; reference 47 cites the matching Kaggle dataset.'},
{'title': 'Jayden P. Chen, Fusion of Neural Networks and Logistic Regression for Predictive Maintenance
```

```
of Vehicle Engines (2024), Table 1, page 66', 'url': 'https://ajosr.org/wp-
content/uploads/journal/published_paper/volume-2/issue-2/ajsr2024_pFcUxi32.pdf#page=3', 'finding':
'Describes class 1 as indicating a need for maintenance. Its sensor summary statistics match this
dataset, although its stated row count differs.'}]}
splits = Path('artifacts/splits')
train, test = prepare(cfg, splits)
print('Label interpretation:', cfg['label_note'])
print('Training shape:', train.shape, 'Test shape:', test.shape)
print('No features dropped; no missing values or exact duplicates in the supplied file.')
```

Code cell 3

```
"""Training-only exploratory analysis and held-out diagnostic plots."""
from pathlib import Path
import json
import joblib
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from IPython.display import display
from sklearn.metrics import RocCurveDisplay, PrecisionRecallDisplay


def explore(train, cfg, output):
    output = Path(output)
    output.mkdir(parents=True, exist_ok=True)
    target = cfg['target']
    features = train.drop(columns=target).columns.tolist()
    sns.set_theme(style='whitegrid', font_scale=0.95)
    summary = train[features].describe().T
    q1, q3 = summary['25%'], summary['75%']
    iqr = q3 - q1
    outlier_counts = ((train[features] < q1 - 1.5*iqr) | (train[features] > q3 + 1.5*iqr)).sum()
    summary['IQR flags'] = outlier_counts
    groups = train.groupby(target)[features].mean().T
    groups.columns = ['Class 0 mean', 'Class 1 mean']
    groups['Difference (1 - 0)'] = groups['Class 1 mean'] - groups['Class 0 mean']
    corr = train.corr()
    summary.to_csv(output / 'feature_summary.csv')
    groups.to_csv(output / 'class_comparison.csv')
    corr.to_csv(output / 'correlations.csv')
    print('Table 1. Training feature overview; pressure/temperature units were not supplied.')
    display(summary.round(2))
    print('Table 2. Training feature means by class (associations, not causal effects).')
    display(groups.round(2))
    print('Table 3. Training missing values and IQR outlier flags (flags are retained).')
    display(pd.DataFrame({'Missing': train[features].isna().sum(), 'IQR flags': outlier_counts}))

    def finish(fig, number):
        fig.tight_layout()
        fig.savefig(output / f'figure_{number}.png', dpi=180, bbox_inches='tight')
        plt.show()
        plt.close(fig)

    fig, ax = plt.subplots(figsize=(7, 3.4))
    counts = train[target].value_counts().sort_index()
    ax.bar(['Class 0', 'Class 1'], counts, color=['#536d91', '#25a399'])
    for i, count in enumerate(counts):
        ax.text(i, count, f'{count:,} ({count/len(train):.2%})', ha='center', va='bottom')
    ax.set(ylim=(0, counts.max()*1.18), xlabel='Engine Condition', ylabel='Training observations',
           title='Figure 1. Training class distribution')
    finish(fig, 1)

    fig, axes = plt.subplots(2, 3, figsize=(11, 6))
    for col, ax in zip(features, axes.flat):
        ax.hist(train[col], bins=35, color='#536d91', edgecolor='white')
        ax.set(xlabel=col, ylabel='Observations')
    fig.suptitle('Figure 2. Sensor distributions (training data)', fontsize=14)
    finish(fig, 2)

    fig, axes = plt.subplots(2, 3, figsize=(11, 6))
    for col, ax in zip(features, axes.flat):
        sns.boxplot(data=train, x=target, y=col, ax=ax, color='#8ac9c2', fliersize=2)
        ax.set(xlabel='Engine Condition class', ylabel=col)
    fig.suptitle('Figure 3. Sensor distributions by class (training data)', fontsize=14)
    finish(fig, 3)

    fig, ax = plt.subplots(figsize=(9, 6.6))
    sns.heatmap(corr, annot=True, fmt='.2f', cmap='vlag', center=0, vmin=-1, vmax=1,
                ax=ax, annot_kws={'size':10})
    ax.set_title('Figure 4. Multivariate Pearson correlations (training data)')
    finish(fig, 4)

    target_corr = corr[target].drop(target)
    strongest = target_corr.abs().idxmax()
    upper_triangle = np.triu(np.ones((len(features), len(features)), dtype=bool), k=1)
    feature_corr = corr.loc[features, features].where(upper_triangle).stack()
    pair = feature_corr.abs().idxmax()
```

```
    rpm_means = groups.loc['Engine rpm']
    coolant = summary.loc['Coolant temp']
    notes = [
        f'Class 1 represents {100*train[target].mean():.2f}% of training observations. A majority-class
prediction can therefore appear accurate while never recognizing class 0; balanced accuracy and both-
class diagnostics are needed.',
        f'Engine rpm averages {rpm_means.iloc[0]:.2f} for class 0 and {rpm_means.iloc[1]:.2f} for class
1. The distributions overlap, so rpm alone should not be used as a maintenance rule.',
        f'{strongest} has the strongest absolute linear association with the target (r =
{target_corr[strongest]:.2f}). This describes an association with the encoded class, not a cause of
engine condition.',
        f'The strongest sensor-to-sensor correlation is {pair[0]} versus {pair[1]} (r =
{feature_corr[pair]:.2f}). Weak pairwise correlations do not rule out nonlinear relationships or
interactions.',
        f'Coolant temp has a median of {coolant["50%"]:.2f} and a maximum of {coolant["max"]:.2f};
{int(outlier_counts["Coolant temp"])} readings lie outside its 1.5-IQR limits. Retain these readings
because they may describe unusual operating conditions; verify units and sensor calibration before
applying physical limits.',
        'No identifier, timestamp, or failure-mode field is present. All six sensor features are
retained. These measurements support contemporaneous condition classification; the data cannot establish
advance warning time or remaining useful life.'
    ]
    (output / 'eda_insights.json').write_text(json.dumps(notes, indent=2))
    for note in notes:
        print(note)
    return notes


def diagnostics(test, cfg, model_dir, output):
    output = Path(output)
    model = joblib.load(Path(model_dir) / 'best_model.joblib')
    metrics = json.loads((Path(model_dir) / 'metrics.json').read_text())
    y = test[cfg['target']]
    X = test.drop(columns=cfg['target'])
    probability = model.predict_proba(X)[:, 1]
    fig, ax = plt.subplots(figsize=(6, 4))
    sns.heatmap(metrics['random_forest']['confusion_matrix'], annot=True, fmt='d', cmap='Blues',
                xticklabels=['Class 0','Class 1'], yticklabels=['Class 0','Class 1'], ax=ax)
    ax.set(xlabel='Predicted class', ylabel='Actual class', title='Figure 5. Held-out confusion matrix')
    fig.tight_layout(); fig.savefig(output / 'figure_5.png', dpi=180); plt.show(); plt.close(fig)
    fig, axes = plt.subplots(1, 2, figsize=(11, 4))
    RocCurveDisplay.from_predictions(y, probability, ax=axes[0], name='Random forest')
    axes[0].plot([0,1],[0,1], '--', color='grey', label='Chance'); axes[0].legend()
    PrecisionRecallDisplay.from_predictions(y, probability, ax=axes[1], name='Random forest')
    axes[1].axhline(y.mean(), linestyle='--', color='grey', label='Class 1 prevalence'); axes[1].legend()
    fig.suptitle('Figure 6. Held-out ranking performance (positive class = 1)')
    fig.tight_layout(); fig.savefig(output / 'figure_6.png', dpi=180); plt.show(); plt.close(fig)
    print('Table 5. Held-out metrics (positive class = 1).')
    display(pd.DataFrame({name:{k:v for k,v in metrics[name].items() if k!='confusion_matrix'}
                          for name in ['baseline','random_forest']}).round(2))
    print(f"Held-out balanced accuracy: {metrics['random_forest']['balanced_accuracy']:.2f}; "
          f"baseline: {metrics['baseline']['balanced_accuracy']:.2f}. "
          f"Class-1 F1: {metrics['random_forest']['f1']:.2f}; baseline:
{metrics['baseline']['f1']:.2f}.")
    print('Selection optimized cross-validated average precision, not F1 at threshold 0.5. '
          'The majority baseline has class-1 recall of 1.00 but recognizes no class-0 observations. '
          'The forest trades some class-1 recall for class-0 recognition; threshold choice needs
validation data and operational costs.')

notes = explore(train, cfg, Path("reports"))
```

Code cell 4

```
metrics = train_model(cfg, splits, Path('artifacts/model'))
print('Table 4. Full experiment log, including all fold scores and tuned parameters.')
experiments = pd.read_csv('artifacts/model/experiments.csv')
with pd.option_context('display.max_columns', None, 'display.max_rows', None):
    display(experiments)
diagnostics(test, cfg, Path('artifacts/model'), Path('reports'))
print('Environment and dataset lineage:')
print(Path('artifacts/model/run_metadata.json').read_text())
```

Appendix D. Workflow artifact handoff configuration

The following GitHub Actions workflow uploads the saved training/test split artifact from the preparation job and downloads it in the dependent training job. The fitted model and experiment logs are then uploaded as a second artifact. The final persist-model job downloads those results into models/ and commits them to the main branch.

GitHub Actions run 41421706788 completed successfully. The prepare job validated the data and uploaded the splits; train downloaded those splits, tuned the model and uploaded the results; persist-model committed the fitted model and logs to models/ (commit ba0ee8a). Run: https://github.com/harshitrathi14/capstone-predictivemaintenance/actions/runs/41421706788. The report and notebook retain the original local execution results; the hosted run has its own metrics and environment record in models/.

Source: .github/workflows/pipeline.yml. The workflow runs on relevant pushes to main and can also be started manually. The automated model commit changes only models/, which is outside the push path filters, so it does not start another training run.

```
name: Predictive maintenance
on:
  workflow_dispatch:
  push:
    branches: [main]
    paths:
      - 'data/**'
      - 'src/pipeline.py'
      - 'tests/**'
      - 'config.json'
      - 'requirements.txt'
      - '.github/workflows/pipeline.yml'
permissions:
  contents: read
concurrency:
  group: predictive-maintenance-${{ github.ref }}
  cancel-in-progress: true
jobs:
  prepare:
    runs-on: ubuntu-latest
    timeout-minutes: 15
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.12'
          cache: pip
      - name: Install dependencies
        run: pip install -r requirements.txt
      - name: Verify pipeline behavior
        run: python -m unittest discover -s tests -v
      - name: Validate data and save train/test splits
        run: python src/pipeline.py prepare
      - uses: actions/upload-artifact@v4
        with:
          name: train-test-splits
          path: artifacts/splits/
          if-no-files-found: error
          retention-days: 90
  train:
    needs: prepare
    runs-on: ubuntu-latest
    timeout-minutes: 20
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.12'
          cache: pip
      - name: Install dependencies
        run: pip install -r requirements.txt
      - uses: actions/download-artifact@v4
        with:
          name: train-test-splits
          path: artifacts/splits/
      - name: Tune and evaluate using downloaded split artifacts
        run: python src/pipeline.py train
      - name: Record workflow provenance
        run: |
          python - <<'PY'
          import json, os
          from pathlib import Path
          info = {key: os.environ[key] for key in ['GITHUB_REPOSITORY', 'GITHUB_SHA', 'GITHUB_RUN_ID',
```

```
'GITHUB_RUN_ATTEMPT']}
          info['run_url'] =
f"https://github.com/{info['GITHUB_REPOSITORY']}/actions/runs/{info['GITHUB_RUN_ID']}"
          Path('artifacts/model/workflow_run.json').write_text(json.dumps(info, indent=2))
          PY
      - uses: actions/upload-artifact@v4
        with:
          name: model-and-experiments
          path: artifacts/model/
          if-no-files-found: error
          retention-days: 90
  persist-model:
    needs: train
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    timeout-minutes: 5
    permissions:
      contents: write
    steps:
      - uses: actions/checkout@v4
        with:
          ref: main
      - uses: actions/download-artifact@v4
        with:
          name: model-and-experiments
          path: models/
      - name: Commit trained model and experiment evidence
        run: |
          git config user.name 'github-actions[bot]'
          git config user.email '41898282+github-actions[bot]@users.noreply.github.com'
          git add models/
          if ! git diff --cached --quiet; then
            git commit -m "Save trained model and experiments from run ${GITHUB_RUN_ID}"
            git push origin HEAD:main
          fi
```